The Van der Pol Oscillator: Comparing Numerical and Machine Learning Approaches

Daniel Shamsoddini
Northwestern University

I. Introduction

The Van Der Pol oscillator is a system represented by a second order Ordinary Differential Equation. It is an oscillating system with non-linear damping and non-conservative oscillations where energy is added to small oscillations and removed from large oscillations. It lacks an analytical solution and can be represented by the equation.

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0 \tag{1}$$

Interestingly, when the coefficient $\mu = 0$, the equation reduces to a simple harmonic oscillator. Variants of it can be found across fields, such as in certain RLC circuits, biological systems, and other nonlinear oscillating systems. The well known Fitzhugh-Nagumo model of neuronal activation is a variant of the Van Der Pol oscillator modified to model excitation and recovery processes unique to neurons.

The Van Der Pol oscillator also has a forced variant, where an external periodic forcing term is present in the system, represented as

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x - A\sin(\omega t) = 0 \tag{2}$$

which makes it a non-autonomous system dependent on time, leading to more complex and potentially chaotic behavior.

In this report I will document my attempt to model both the unforced and forced Van Der Pol oscillators using both Numerical and Scientific Machine Learning based approaches, my methods in doing so, and their corresponding results along with my thoughts and suggestions for anyone seeking to do the same.

II. Methods

A. Numerical

Since the Van Der Pol oscillator is a second order ODE and can be written as a system of two first order ODEs

$$x' = y \tag{3}$$

$$y' = \mu(1 - x^2)y - x \tag{4}$$

$$y' = \mu(1 - x^2)y - x + A\sin(\omega t) \tag{5}$$

with equations (3,4) for the unforced and (3,5) for the forced variants, we can attempt to use numerical methods to approximate their solutions.

One such method is the forward Euler method, which I elected to use due to its simplicity and intelligibility, as well as the fact that it is commonly used to time-step Neural ODEs, one of the ML methods I will be discussing below. As a comparison method, I elected to use the fourth order Runge–Kutta method (RK4), due to it being a much more accurate method given the same Δt, as well as also being an explicit numerical method. I then ran three iterations of numerical methods, the forward Euler and RK4 methods with a timestep of $\Delta t = 0.025$, and an RK4 iteration with a lower timestep of $\Delta t = 0.005$ to serve as a reference and evaluate the two methods.

B. Machine Learning

Alternatively to numerical methods, in cases where the underlying differential equation is unknown, or a lengthy integration process is computationally expensive, several machine learning approaches can be used to learn the system dynamics.

The simplest machine learning approach I used was a feedforward neural network trained to map $t \to x$, with y recovered via automatic differentiation. This method is computationally efficient and easy to train, but does not incorporate physical knowledge and thus might diverge outside the training interval. Another method I explored was the Neural Ordinary Differential Equation (Neural ODE), where a neural network represents the system $\dot{z} = f_\theta(z)$, and trajectories are obtained by integrating this learned vector field using a numerical solver. Neural ODEs can capture system behavior more consistently over longer time horizons and attempt to learn the underlying differential equation without requiring prior knowledge beyond assuming an ODE representation. However, they require an ODE solver during training, lack explicit physical constraints, and are more computationally expensive than ANNs. The third method I tried was the Physics-Informed Neural Network (PINN), traditionally used for solving partial differential equations but also usable with ODEs. Similar to the FFNN, a neural network is trained to map $t \to x$, with the addition of a physics or differential equation–based loss. This allows it to require less or even no sampled data while enforcing consistency with physical or system dynamics. However, it requires knowledge of the underlying equation and can be slow and sensitive to hyperparameters and boundary conditions. In this case, since the differential equations are known, the system dynamics can be directly enforced during training.

I thus used these three methods in order to compare the ability of different Machine Learning methods to model the system dynamics of the Unforced and Forced Van Der Pol oscillators. They were trained on data with centered gaussian

noise ($\sigma = 0.1$) and in the case of the PINN, 3000 collocation points from $[0, 100]$ or $[0, 1000]$ per epoch. The parameters of the oscillators I trained were $\mu = 2.0$, $t = [0, 20]$, $z(0) = [1.0, 0.0]$. All models possessed equal neuron layer widths of 128, with the ANN having 3 hidden layers, the Neural ODE having 2, and the PINN having 6 hidden layers. Training was done for 2000 epochs for the FFNN and PINN and 5000 epochs for the Neural ODE, with a learning rate of 5×10^{-4}. For the forced VDP, time t was added as an input parameter to the Neural ODE, in order to account for the non-autonomous nature of the system. The forward Euler method was used to generate the trajectory rollouts during training and evaluation for the Neural ODE due to computational resource constraints, as training it is significantly more computationally taxing than the other two methods. I did not test a Lagrangian or Hamiltonian neural network because of the fact that they are primarily suited for conservative systems, while both Van der Pol oscillators are non-conservative.

C. Results

After solving both oscillators with the forward Euler and RK4 methods,

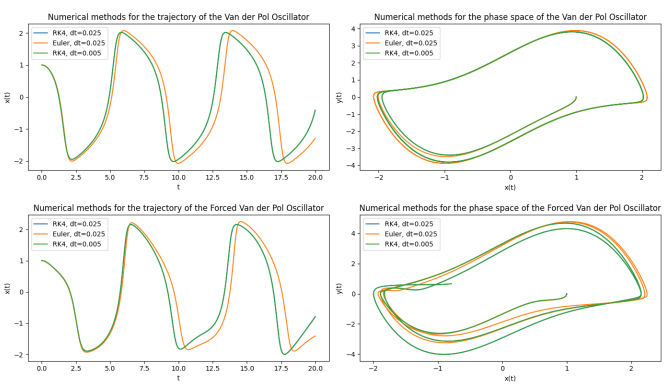

Fig. 1. Numerical Methods

We can see that the explicit Euler method error is noticeable compared to the RK4 for the forced VDP oscillator in both trajectory and phase space, while for the unforced VDP, holding all parameters equal, this difference is less noticeable, but still present. Interestingly, there is almost no visibly meaningful difference between the RK4 runs with timesteps of $\Delta t = 0.025$ and $\Delta t = 0.005$.

For the Machine Learning based approaches, the results we get are somewhat more disappointing, however still insightful. Within the training interval, all of the ML methods are able to approximate to their reference low-timestep RK4 from which data was sampled relatively well, with the ANN and PINN being roughly equivalent and ahead of the Neural ODE in this regard. However, when it came to the time period beyond the training interval, the ANN quickly collapses towards zero as expected, but the PINN surprisingly also does so despite my decision to allow the collocation points to be sampled from beyond the training domain, regardless of whether on the interval $[0, 100]$ or $[0, 1000]$. The Neural ODE as expected performs the best past the training interval but even it exhibits some drift and increasing deviation over time. For the forced VDP oscillator, we can then see that the results are mostly identical to the unforced VDP, albeit noticeably worse for the phase space plots and for the Neural ODE past $t = 20$, despite it being still the best performing.

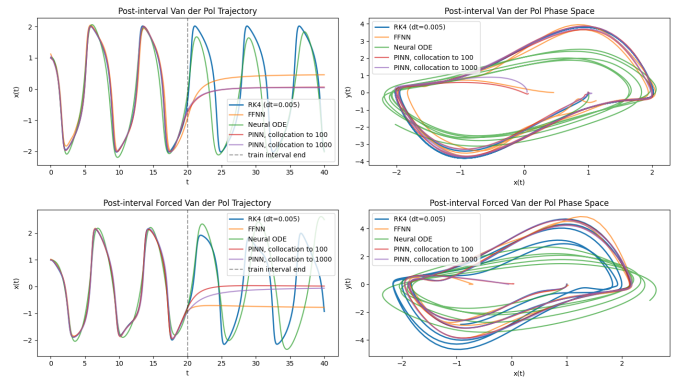

Fig. 2. Machine Learning Methods

III. Discussion

The results demonstrate that while machine learning methods are capable of learning the Van Der Pol oscillator within a training interval from sampled data, they struggle greatly with extrapolating past their training interval. The feedforward neural network and the Physics Informed Neural Network both perform well in the training interval, but fail to generalize beyond it. In particular, they both completely collapse toward a constant value, highlighting their lack of understanding of the underlying system dynamics visible in the phase space plots in the case of the FFNN and the failure of collocation points to enforce them past the training interval in the case of the PINN. The Neural ODE does better with its learning a vector field representation of the system, which allowed it to preserve oscillatory behavior over longer time horizons. However, it exhibited drift in both trajectory and phase space, indicating that small errors in the learned dynamics accumulate over time and that the vector field learned was not the most robust. These limitations are even more present in the forced Van der Pol oscillator, where the presence of a time-dependent forcing term increased the complexity of the system, leading to further deteriorated performance. However, I believe that the Neural ODE method could see greater success for both oscillator types and long time horizons if smaller timesteps or if a higher order integrator such as RK4 was used.

Overall, this project has shown me that while machine learning approaches are useful for modeling dynamical systems, and can succeed on known intervals, they struggle with extrapolation past those time intervals, especially in nonlinear and forced systems like the Van Der Pol oscillators. Numerical methods such as RK4 remain much more reliable for accurate trajectory prediction, while machine learning methods are better suited for short-term approximation or situations where the underlying differential equations are unknown. However, I ultimately believe that they can be of use for those with increased computational resources and those who are able to tune the model architectures and training methods they use.